



FUNDAMENTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

Informe Técnico — Proyecto Final: Bus Escape: Traffic Jam

Juan Diego López Gómez | Código: 2519861 | juan.lopez.g@correounivalle.edu.co
Douglas Vélez Monginaite | Código: 2516737 | douglas.velez@correounivalle.edu.co
Nicolás Andrés Ramírez Correa | Código: 2415510 | nicolas.andres.ramirez@correounivalle.edu.co

1. Introducción

El presente informe documenta el desarrollo del proyecto *Bus Escape: Traffic Jam*, un juego de lógica implementado en C++ que simula la gestión de tráfico y pasajeros en un tablero congestionado. El software fue construido aplicando los cuatro pilares de la Programación Orientada a Objetos (POO) y los principios SOLID, bajo una arquitectura estricta del patrón MVC (Modelo-Vista-Controlador).

El objetivo del jugador es mover estratégicamente los vehículos para que recojan a los pasajeros que esperan en fila, validando la coincidencia de colores. El juego carga su configuración inicial desde archivos planos (.txt) y se ejecuta completamente en consola.

2. Documentación CRC

La documentación CRC (Class-Responsibility-Collaboration) describe cada clase del sistema indicando su nombre, su responsabilidad única y las clases con las que colabora para cumplir su función dentro de la arquitectura MVC.

2.1 Capa Modelo

Clase	Responsabilidad	Colaboración
Vehicle	Definir el contrato base para todos los vehículos. Gestionar posición, color, orientación, longitud, conteo de pasajeros y capacidad máxima.	Bus, Car (subclases), Board (almacena vehículos), ParkingZone (recibe vehículos), PassengerQueue (pasajeros que abordan).
Bus	Representar un bus. Especializar la visualización (display) y el tipo. Capacidad igual a su longitud.	Vehicle (hereda), GameController (instancia y gestiona), Board (posiciona).

Car	Representar un carro. Añadir atributo attachedToBus para futura extensión. Capacidad igual a su longitud.	Vehicle (hereda), GameController (instancia y gestiona), Board (posiciona).
Passenger	Modelar a un pasajero individual con ID y color. El color actúa como restricción de abordaje.	PassengerQueue (almacena pasajeros), GameController (valida coincidencia de color con vehículo).
Passenger-Queue	Administrar la fila FIFO de pasajeros. Encapsular std::queue y exponer operaciones seguras de encolado y desencolado.	Passenger (contiene), GameController (consulta el frontal para validar abordaje), View (muestra la cola).
ParkingZone	Gestionar el estado de una celda de parqueo. Validar que solo un vehículo ocupe la zona. Detectar vehículos llenos.	Vehicle (aloja), GameController (envía/retira vehículos), View (muestra estado de ocupación).
Board	Gestionar el espacio físico del juego (grid). Detectar colisiones, validar movimientos, verificar salidas del tablero, actualizar posiciones y renderizar el grid.	Vehicle (almacena y posiciona), GameController (solicita movimientos y validaciones), View (muestra el tablero).
LevelData	Actuar como contenedor plano de la configuración de un nivel: dimensiones, zonas, vehículos y pasajeros.	LevelLoader (llena la estructura), GameController (usa para inicializar y reiniciar niveles).
LevelLoader	Leer archivos .txt y construir objetos internos (vehículos y pasajeros) dentro de una estructura LevelData.	LevelData (llena), Bus, Car, Passenger (instancia), GameController (solicita la carga).

2.2 Capa Vista

Clase	Responsabilidad	Colaboración
View	Renderizar visualmente el estado del juego en consola. Integrar tablero, zonas de parqueo y fila de pasajeros en un único fotograma. Mostrar mensajes de error, éxito e información.	Board (consulta estado), ParkingZone (consulta ocupación), PassengerQueue (consulta pasajeros), GameController (solicita renderizado).
Menu	Gestionar toda la interacción textual con el usuario. Presentar menús principales, de selección de vehículos y de acciones. Capturar y validar entrada por teclado.	Vehicle (lista para selección), GameController (coordina las opciones elegidas), View (complementaria).

2.3 Capa Controlador

Clase	Responsabilidad	Colaboración
GameController	Orquestrar el flujo completo de la partida. Coordinar Modelo, Vista y Menú. Implementar reglas de negocio: abordaje automático, movimiento, parqueo, detección de victoria/derrota, progresión de niveles y reinicio.	Board (solicita movimientos), ParkingZone (gestiona parqueo), PassengerQueue (valida abordaje), View (solicita renderizado), Menu (recibe entrada), LevelLoader/LevelData (carga niveles), Vehicle (polimórfico).

2.4 Enumeraciones

Clase/Enum	Responsabilidad	Colaboración
Orientation	Definir los dos valores posibles de orientación de un vehículo: HORIZONTAL y VERTICAL.	Vehicle (atributo), Board (lógica de colisiones y movimiento depende de la orientación).

3. Requerimientos del Sistema

3.1 Historias de Usuario

Código	Descripción
HU-01	Cargar un nivel desde archivo. Como jugador, quiero que el juego cargue la configuración inicial desde un archivo .txt para poder jugar niveles predefinidos con distintas dificultades.
HU-02	Ver el tablero en consola. Como jugador, quiero ver el tablero con los vehículos identificados visualmente para poder planificar mis movimientos.
HU-03	Seleccionar y mover un vehículo. Como jugador, quiero seleccionar un vehículo por su ID y moverlo hasta la zona de embarque para intentar recoger pasajeros.
HU-04	Validar coincidencia de colores. Como jugador, quiero que el sistema valide automáticamente si el color del vehículo coincide con el pasajero al frente de la fila, para que solo suban los pasajeros correctos.
HU-05	Gestionar zonas de parqueo. Como jugador, quiero que los vehículos llenos se retiren de las zonas de parqueo para liberar espacio y continuar el juego.
HU-06	Detectar derrota. Como jugador, quiero que el juego me avise cuando las 4 zonas de parqueo estén llenas y no sea posible continuar, para saber que perdí la partida.
HU-07	Detectar victoria. Como jugador, quiero que el juego me avise cuando todos los pasajeros hayan abordado un vehículo, para saber que completé el nivel.
HU-08	Reiniciar o avanzar de nivel. Como jugador, quiero poder reiniciar el nivel actual o avanzar al siguiente nivel para seguir jugando.

3.2 Criterios de Aceptación

Historia	Criterios de Aceptación
HU-01	El archivo .txt contiene dimensiones del tablero, definición de vehículos (tipo, color, X, Y, orientación, largo) y cola de pasajeros por color. Board::cargarDesdeArchivo() construye objetos Vehicle (Bus/Car) correctamente. GameController::initializeLevel(levelNumber) carga el archivo correspondiente. Si el archivo no existe o está mal formado, el sistema lanza un error controlado (no crash).

HU-02	View::displayGameBoard() muestra todas las celdas del grid con los IDs de los vehículos. View::displayPassengerQueue() muestra la fila de pasajeros con sus colores. View::displayParkingZones() muestra el estado de las 4 zonas de parqueo. El tablero se actualiza tras cada movimiento.
HU-03	Board::canVehicleMove() verifica que el camino esté libre antes de mover. Board::hasCollision() detecta colisiones con otros vehículos. Un vehículo solo puede moverse en su eje fijo (horizontal o vertical). Menu::showVehicleSelectionMenu() lista solo los vehículos disponibles en el tablero.
HU-04	GameController::validatePassengerBoarding() compara el color del vehículo con PassengerQueue::frontPassenger(). Si coinciden, se ejecuta boardPassenger() que llama a Vehicle::addPassenger() y PassengerQueue::removePassenger(). Si no coinciden, el sistema informa al jugador sin abordar ningún pasajero. El sistema respeta el orden estricto de la fila (FIFO).
HU-05	Solo existen 4 instancias de ParkingZone. ParkingZone::parkVehicle() solo acepta un vehículo si isOccupied == false. GameController::removeFullVehicles() detecta vehículos llenos (Vehicle::isFull()) y los retira. ParkingZone::isVehicleFull() confirma que el vehículo ya no puede recibir más pasajeros.
HU-06	GameController::isParkingFull() retorna true cuando las 4 zonas están ocupadas. GameController::hasPlayerLost() retorna true si no hay zona libre y aún quedan pasajeros en la fila. El sistema muestra un mensaje de derrota claro en consola.
HU-07	GameController::hasPlayerWon() retorna true cuando PassengerQueue::isEmpty() == true. GameController::hasGameEnded() encapsula ambas condiciones (victoria y derrota). El sistema ofrece avanzar al siguiente nivel si hasNextLevel() es verdadero.
HU-08	GameController::resetLevel() restaura el estado inicial guardado en currentLevelData. GameController::initializeLevel(n+1) carga el siguiente archivo de nivel. El menú principal ofrece las opciones: Jugar, Instrucciones, Salir.

4. Descripción de Clases y Responsabilidades

El sistema está organizado en tres capas arquitectónicas: **Modelo**, **Vista** y **Controlador**. A continuación se describe cada clase, su propósito y su responsabilidad única dentro del diseño.

4.1 Capa Modelo

Vehicle (clase abstracta)

Responsabilidad: Establecer el contrato base para todos los vehículos del juego. Gestiona atributos universales: posición (x, y), longitud, color, orientación, conteo de pasajeros y capacidad máxima. Define métodos virtuales puros `display()` y `getType()` que las subclases deben implementar. Aplica **abstracción** y sirve como raíz de la jerarquía polimórfica.

Bus (hereda de Vehicle)

Responsabilidad: Representar un bus dentro del juego. Su capacidad máxima equivale a su longitud. Implementa `display()` para mostrar su estado en consola y `getType()` retornando "Bus". No añade atributos propios, pero especializa el comportamiento visual del vehículo.

Car (hereda de Vehicle)

Responsabilidad: Representar un carro (vehículo pequeño). Añade el atributo `attachedToBus` para modelar la posibilidad de acoplamiento futuro. Implementa `display()` y `getType()` con la etiqueta "Car". Su capacidad máxima equivale a su longitud (generalmente 2).

Passenger

Responsabilidad: Modelar a un pasajero individual. Almacena un identificador numérico y un color que actúa como restricción de abordaje. Es una entidad simple, inmutable en su color, que viaja a través de la cola de espera hacia los vehículos.

PassengerQueue

Responsabilidad: Administrar la estructura FIFO (First-In, First-Out) de pasajeros en espera. Encapsula un `std::queue<Passenger>` y expone operaciones seguras: `addPassenger()`, `frontPassenger()`, `removePassenger()`, `isEmpty()` y `getSize()`. Aísla la implementación subyacente del resto del sistema.

ParkingZone

Responsabilidad: Gestionar el estado de una celda de parqueo individual. Contiene un `shared_ptr<Vehicle>` opcional y un flag `occupied`. Valida que solo un vehículo ocupe la zona a la vez (`parkVehicle()`) y detecta cuando un vehículo estacionado alcanzó su capacidad máxima (`isVehicleFull()`).

Board

Responsabilidad: Gestionar el espacio físico del juego (grid). Mantiene las dimensiones del tablero y la colección de vehículos activos. Implementa la lógica de colisiones mediante solapamiento de rectángulos (`hasCollision()`), valida límites de movimiento (`canVehicleMove()`), verifica si un vehículo puede salir del tablero (`canVehicleExitBoard()`) y actualiza posiciones (`moveVehicle()`). También renderiza el grid en consola (`mostrar()`).

LevelData (struct)

Responsabilidad: Actuar como contenedor de datos plano para la configuración de un nivel. Almacena dimensiones del tablero, número de zonas de parqueo, lista de vehículos y lista de pasajeros. Es usado por `LevelLoader` y por `GameController` para restaurar el estado inicial en reinicios.

LevelLoader

Responsabilidad: Manejar la persistencia del juego. Lee archivos `.txt` con formato definido y construye objetos `Bus`, `Car` y `Passenger` dentro de una instancia `LevelData`. Aplica el principio **SRP**: solo lee archivos, sin lógica de juego ni impresión en consola.

4.2 Capa Vista

View

Responsabilidad: Renderizar visualmente el estado completo del juego en consola. Integra el tablero, las zonas de parqueo y la fila de pasajeros en un único fotograma (`displayGameStatus()`). También limpia la pantalla (`clearScreen()`) y muestra mensajes de error, éxito o información general. No contiene lógica de negocio.

Menu

Responsabilidad: Gestionar toda la interacción textual con el usuario. Presenta el menú principal, el menú de selección de vehículos y el menú de acciones por vehículo. Captura entrada por teclado con validación de rangos y recuperación de errores. Incluye utilidades como `pause()` para sincronizar la lectura del usuario.

4.3 Capa Controlador

GameController

Responsabilidad: Orquestrar el flujo completo de la partida. Coordina las interacciones entre Modelo, Vista y Menú. Implementa las reglas de negocio: boarding automático de pasajeros (`processAutoBoarding()`), movimiento de vehículos (`handleMoveVehicle()`), envío a parqueo (`handleParkVehicle()`), detección de victoria (`hasPlayerWon()`) y derrota (`hasPlayerLost()`). Gestiona la progresión de niveles y el reinicio del estado. Es el único componente que conoce a las tres capas MVC.

5. Aplicación de los Cuatro Pilares de la POO

5.1 Abstracción

Se identificaron las entidades fundamentales del dominio (vehículos, pasajeros, tablero, zonas de parqueo) y se modelaron como clases con atributos y comportamientos propios. La clase `Vehicle` abstrae las características comunes de todos los vehículos (posición, color, orientación, capacidad), delegando la representación específica a las subclases `Bus` y `Car`.

5.2 Encapsulamiento

Todos los atributos de las clases se declararon como `private` o `protected`, exponiendo únicamente los métodos necesarios mediante `public`. Por ejemplo, `PassengerQueue` oculta la estructura interna `std::queue` y solo permite operaciones controladas: `addPassenger()`, `frontPassenger()` y `removePassenger()`.

5.3 Herencia

Se estableció una jerarquía de herencia donde `Bus` y `Car` heredan de la clase base abstracta `Vehicle`. Esto permite tratar ambos tipos de forma polimórfica: el tablero almacena `std::shared_ptr<Vehicle>` sin distinguir si se trata de un bus o un carro hasta el momento de invocar métodos virtuales como `display()` o `getType()`.

5.4 Polimorfismo

El polimorfismo se aplica en múltiples capas del sistema:

- **Polimorfismo de inclusión:** `Vehicle` define métodos virtuales puros (`display()`, `getType()`) que `Bus` y `Car` implementan.
- **Polimorfismo paramétrico:** Uso de templates de la STL (`std::vector`, `std::queue`, `std::shared_ptr`).
- **Polimorfismo de subtipado:** `ParkingZone::parkVehicle(std::shared_ptr<Vehicle>)` acepta cualquier subclase de `Vehicle`.

6. Aplicación de los Principios SOLID

S — Single Responsibility Principle (SRP)

Cada clase tiene una única razón para cambiar. `LevelLoader` se encarga exclusivamente de la persistencia (lectura de archivos `.txt`); `View` solo renderiza en consola; `Board` solo gestiona colisiones y posiciones. El controlador (`GameController`) orquesta, pero no implementa lógica de bajo nivel.

O — Open/Closed Principle (OCP)

El sistema está abierto a extensión pero cerrado a modificación. Si en el futuro se requiere un nuevo tipo de vehículo (ej. `Truck`), basta con crear una clase que herede de `Vehicle` e implemente sus métodos virtuales. No es necesario modificar `Board`, `ParkingZone` ni `GameController`, ya que todas operan sobre la abstracción `Vehicle`.

L — Liskov Substitution Principle (LSP)

`Bus` y `Car` pueden sustituir a `Vehicle` sin alterar el comportamiento correcto del programa. Por ejemplo, `Board::addVehicle(std::shared_ptr<Vehicle>)` funciona indistintamente con buses o carros, y `ParkingZone::parkVehicle()` no necesita verificar el tipo concreto.

I — Interface Segregation Principle (ISP)

La interfaz de `Vehicle` es mínima y cohesiva: solo expone operaciones relevantes para cualquier vehículo (posición, color, pasajeros, capacidad). No obliga a las subclases a implementar métodos que no necesitan. `Car` añade `attachToBus()` como método propio, sin afectar la interfaz base.

D — Dependency Inversion Principle (DIP)

Las clases de alto nivel dependen de abstracciones, no de implementaciones concretas.

`GameController` depende de `Vehicle` (abstracta), no de `Bus` o `Car`. `Board` almacena `std::shared_ptr<Vehicle>`, no objetos concretos. Esto desacopla el modelo del controlador.

7. Arquitectura MVC

Capa	Clases	Responsabilidad
Modelo	Vehicle, Bus, Car, Passenger, PassengerQueue, Board, ParkingZone, LevelData, LevelLoader	Gestión de datos, reglas de negocio, persistencia y estado del juego.
Vista	View, Menu	Representación visual en consola, captura de entrada del usuario y navegación por menús.
Controlador	GameController	Orquestación del flujo, validación de reglas, coordinación entre Modelo y Vista, detección de fin de partida.

8. Pruebas Unitarias

Se implementaron pruebas automatizadas con GoogleTest (`tests/test_main.cpp`) para verificar el correcto funcionamiento de los componentes críticos:

- Detección de colisiones (`Board::hasCollision`)
- Movimiento de vehículos libres y bloqueados (`Board::canVehicleMove`)
- Polimorfismo de vehículos (`Bus` vs `Car`)
- Gestión FIFO de pasajeros (`PassengerQueue`)
- Zonas de parqueo (`ParkingZone::parkVehicle`)
- Carga de niveles válidos e inválidos (`LevelLoader::load`)

9. Conclusión

El proyecto *Bus Escape: Traffic Jam* demuestra la aplicación rigurosa de los fundamentos de la Programación Orientada a Objetos y los principios SOLID. La separación en capas MVC, el uso de herencia y polimorfismo, y el diseño modular permiten que el sistema sea mantenible, extensible y fácil de probar. La inclusión de pruebas unitarias, la carga dinámica de niveles desde archivos planos y la gestión automática de pasajeros refuerzan la calidad del software desarrollado, cumpliendo con todos los requerimientos funcionales establecidos en las historias de usuario.